WSL2 Setup Guide — LangConfig

LangConfig on WSL2 (Ubuntu) — native PostgreSQL setup

Verified on WSL2 + Ubuntu with **native** PostgreSQL 16 (no Docker). For the Docker path or macOS, see `docs/TEAM_INSTALL.md` instead.

What you end up with

Service	URL
Frontend (Vite/React)	http://localhost:1421
Backend API (FastAPI)	http://localhost:8765
Swagger / OpenAPI	http://localhost:8765/docs
PostgreSQL (native)	`localhost:5432` — user `langconfig`, db `langconfig`
Neo4j (optional)	http://localhost:7474 (browser) · `bolt://localhost:7687`

Versions this guide assumes: **Python 3.13**, **Node ≥ 20.19 or ≥ 22.12**, **PostgreSQL 16 + pgvector**.

Contents

Part 1 — install (do once)

1. Base tools · 2. Clone · 3. PostgreSQL 16 + pgvector · 4. Role + database · 5. `.env` ·
2. Python dependencies · 7. Seed · 8. Verify · 9. Frontend dependencies · 10. Start everything ·
3. Neo4j (optional) · 12. Troubleshooting · 13. Daily workflow

Part 2 — WSL platform notes 14. WSL tuning (`/etc/wsl.conf`, `.wslconfig`) · 15. Windows ↔ WSL integration · 16. Docker on WSL · 17. Disk space and the WSL VHDX · 18. Backups and restore · 19. Starting the stack automatically · 20. Playwright browsers · 21. SSH keys for git

> ⚠️ **Put the repo on the Linux filesystem** (`~/langconfig`), **never** on a Windows drive (`/mnt/c/...`). `uv` uses hardlinks and mmap, and the drvfs/9p filesystem produces `bus error` (SIGBUS) during `uv sync`. Disk budget: the venv is ~2 GB and uv's cache can reach several GB.

---

1. Base tools

```
sudo apt update && sudo apt install -y git curl ca-certificates unzip lsof
lsb_release -cs          # 'noble' = 24.04  |  'jammy' = 22.04
```

`lsof` is used by `start.sh` to stop stale processes on ports 8765/1421.

2. Clone

```
cd ~
git clone git@github.com:sailor723/langconfig.git langconfig
# no SSH key on this WSL? use HTTPS instead:
#   git clone https://github.com/sailor723/langconfig.git langconfig

cd langconfig
git log --oneline -1        # expect the current main tip
git status -sb              # expect: ## main...origin/main  (no ahead/behind)
```

3. PostgreSQL 16 + pgvector

Ubuntu 24.04 (`noble`) — packages are in the distro repos

```
sudo apt install -y postgresql-16 postgresql-16-pgvector
```

Ubuntu 22.04 (`jammy`) — needs the official PGDG repo first

Jammy ships PostgreSQL 14 and no pgvector package, so add PGDG:

```
sudo install -d /usr/share/postgresql-common/pgdg
sudo curl -o /usr/share/postgresql-common/pgdg/apt.postgresql.org.asc --fail \
    https://www.postgresql.org/media/keys/ACCC4CF8.asc
sudo sh -c 'echo "deb [signed-by=/usr/share/postgresql-common/pgdg/apt.postgresql.org.asc] \
    https://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg main" > \
    /etc/apt/sources.list.d/pgdg.list'
sudo apt update
sudo apt install -y postgresql-16 postgresql-16-pgvector
```

Quick check: `apt-cache policy postgresql-16 postgresql-16-pgvector` — if both show a real `Candidate:`, you can skip the PGDG step on that machine.

Start the service and verify

```
sudo service postgresql start
pg_isready -h localhost -p 5432      # → localhost:5432 - accepting connections
pg_lsclusters                        # → "16  main  5432  online"
```

> **WSL does not auto-start PostgreSQL.** After every WSL restart, run `sudo service postgresql start` again (or `sudo systemctl start postgresql` if you enabled systemd in `/etc/wsl.conf`).

> ⚠️ If `pg_lsclusters` already shows a **14** cluster on port 5432, it will fight PostgreSQL 16 for the port.

4. Create the role + database (once)

```
sudo -u postgres psql -c "CREATE ROLE langconfig LOGIN PASSWORD 'langconfig_dev' CREATEDB SUPERUSER;"
sudo -u postgres psql -c "CREATE DATABASE langconfig OWNER langconfig;"
```

Both privileges are required:

- `CREATEDB` — the seed creates the `chinook` sample database (step 10)
- `SUPERUSER` — `CREATE EXTENSION vector` is run **as the app user**, and pgvector is not a trusted extension. Without it you get `permission denied to create extension "vector"`. (Letting `seed_all.py` auto-create the role gives it only `CREATEDB`, so create it manually as above.)

Verify:

```
PGPASSWORD=langconfig_dev psql -h localhost -p 5432 -U langconfig -d langconfig -c "select version();"
```

5. Create `.env` at the repo root

There is exactly **one** `.env`, at `~/langconfig/.env` (the backend loads it via `../../.env`). It is gitignored.

```
cd ~/langconfig
cp env_example .env
chmod 600 .env

# required: 64-hex Fernet master key for API keys stored in the DB
sed -i "s|^API_KEY_ENCRYPTION_KEY=.*|API_KEY_ENCRYPTION_KEY=$(openssl rand -hex 32)|" .env
```

Only these matter for a native install:

Variable	Value	Note
`POSTGRES_HOST`	`localhost`	
`POSTGRES_PORT`	**`5432`**	**5432 = native, 5433 = Docker.** The single most common breakage
`POSTGRES_DB` / `POSTGRES_USER` / `POSTGRES_PASSWORD`	`langconfig` / `langconfig` / `langconfig_dev`	
`API_KEY_ENCRYPTION_KEY`	64 hex chars	Set it **before** storing any API key in the UI: changing it later makes stored keys unreadable
`LANGCONFIG_WORKSPACE_ROOT`	`backend/.workspace_data`	Project-level `AGENTS.md` dir — **not** the per-agent root

Do **not** set `DATABASE_URL` — it overrides the whole `POSTGRES_*` block.

Confirm what the app resolves:

```
cd ~/langconfig/backend
uv run python -c "from dotenv import load_dotenv; load_dotenv('../.env'); from db.db_url import
        compose_db_url; print(compose_db_url())"
# → postgresql://langconfig:langconfig_dev@localhost:5432/langconfig
```

6. Python dependencies

```
curl -LsSf https://astral.sh/uv/install.sh | sh
source ~/.local/bin/env          # or restart the shell

cd ~/langconfig/backend
uv sync                          # creates backend/.venv (Python 3.13) — resolves 282 packages
uv run python -c "import fastapi, langgraph, deepagents, sqlalchemy, psycopg, neo4j, openpyxl;
        print('deps OK')"
```

> `uv sync` prints `Resolved 282 packages` and installs only the **delta** — the `Installed N packages` line is not a total. To see the real count: `uv run python -c "import importlib.metadata as m; print(len(list(m.distributions())))"` → ~260. If it reports a SIGBUS: `rm -rf .venv && uv cache clean && UV_LINK_MODE=copy uv sync` (causes: the repo on `/mnt/c`, a full disk, or a corrupt cache — see the note at the top of this file and the `bus error` row in §12).

7. Seed the database

```
cd ~/langconfig/backend
WORKSPACE_ROOT="$HOME/langconfig/backend/outputs" uv run python scripts/seeds/seed_all.py
```

Use your real path for `WORKSPACE_ROOT` (never a `/Users/...` macOS path). It is optional — it only sets the folder the identity backfill bakes into agent prompts, and it is **not** the same thing as `LANGCONFIG_WORKSPACE_ROOT` in `.env`. The seed is idempotent; add `--replace` only to force-reload the tools/agents dumps.

The 10 steps: init server → init db (+pgvector) → alembic stamp head → settings → 28 models → 6 search providers → tools → deep agents → identity backfill (step 8.5) → skills → chinook.

> `[1/10] init_server` should report "*role 'langconfig' already exists*" / "*database 'langconfig' already exists*" — that confirms §4 was done. If instead it prints "*Cannot connect as a superuser*", the role/database were never created.

8. Verify

```
cd ~/langconfig
bash scripts/wsl_check.sh
```

All-green baseline: 48 tables · `alembic_version = 029_add_tool_definition` · `vector 0.8.2` enabled · `settings_documents` 7 · `models` 28 · `custom_tools` 67 · `deep_agent_templates` 51 · `skills` 37 · chinook 11 tables.

(The script is read-only; its ❌ lines are the to-do list.)

9. Frontend dependencies

Vite 7 requires **Node ≥ 20.19 or ≥ 22.12** (`engines: ^20.19.0 || >=22.12.0`). Ubuntu's apt `nodejs` is often too old — use nvm:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
source ~/.nvm/nvm.sh
nvm install 22
node -v                          # v22.x

cd ~/langconfig
npm install                      # REQUIRED — nothing is shipped in git
```

Run `npm install` again after any `git pull` that changes `package.json` / `package-lock.json`; otherwise vite fails on missing modules.

10. Start everything

```
cd ~/langconfig
sudo service postgresql start    # WSL does not auto-start Postgres
bash start.sh                    # db check + backend + frontend
```

Expected:

```
── PostgreSQL ── ✅ PostgreSQL already running on :5432
── Backend ──── ✅ Backend healthy on http://localhost:8765
── Frontend ─── ✅ Frontend running on http://localhost:1421
```

Variants: `bash start.sh --be` (backend only) · `bash start.sh --fe` (frontend only).

Stop everything:

```
kill $(cat /tmp/langconfig-be.pid) $(cat /tmp/langconfig-fe.pid)
```

Verify both ports (vite uses `strictPort: true`, so a busy 1421 makes it exit rather than move to another port):

```
ss -ltnp | grep -E ':(8765|1421)'
curl -s -o /dev/null -w "backend  -> %{http_code}\n" http://127.0.0.1:8765/api/chat/health
curl -s -o /dev/null -w "frontend -> %{http_code}\n" http://localhost:1421/
```

Open http://localhost:1421 — from Windows too (WSL2 forwards `localhost`). Use `localhost`, **not** the WSL IP (`172.x.x.x`): vite's `allowedHosts` only permits `localhost`, so an IP gives `Blocked request. This host is not allowed.`

The backend also serves `GET /` (plain JSON info) and `/docs` (Swagger) — handy for checking the API without the UI, but the UI itself is served **only** by vite on 1421.

> Note: `start.sh` backgrounds the frontend, so a vite failure does not stop the script — you just see "connection refused" on 1421. To see vite's real error run it in the foreground: `cd ~/langconfig && npm run dev`.
>
> On some revisions `start.sh` starts PostgreSQL with `brew` (a macOS command) when the service is not already up, which aborts the script on WSL. Starting Postgres first (as above) avoids that path entirely.

11. Optional — Neo4j (Cypher agent / Graph Studio)

The app runs fully without Neo4j; only those two features need it. (Native WSL Docker, or start Neo4j on another host and point `NEO4J_URI` at it.)

```
docker run -d --name langconfig-neo4j \
  -p 7474:7474 -p 7687:7687 \
  -e NEO4J_AUTH=neo4j/password \
  neo4j:2025.10.1
```

`.env` already points at `bolt://localhost:7687` / `http://localhost:7474`.

Load the HR knowledge graph (community-compatible JSON seed — no license needed):

```
cd ~/langconfig/backend
uv run python scripts/seeds/neo4j-hr/import_hr_seed.py bolt://localhost:7687 neo4j password \
    scripts/seeds/neo4j-hr/hr_nodes.json scripts/seeds/neo4j-hr/hr_rels.json
# → VERIFY: nodes=6978 rels=23281
```

> `neo4j.dump` in the same folder is an **enterprise** dump (block storage format) — it cannot be restored without an enterprise license. Use the JSON seed above.

12. Troubleshooting

Symptom	Cause	Fix
`connection refused` on **8765**	backend not running / crashed	`cd backend && .venv/bin/uvicorn main:app --port 8765` to see the traceback
`connection refused` on **1421**	`npm install` never run, or vite error	`cd ~/langconfig && npm run dev` (foreground) to read the error
`Vite requires Node.js version 20.19+`	old Node from apt	`nvm install 22`
`Error: Port 1421 is already in use`	stale vite (strictPort exits)	`ss -ltnp | grep 1421` → kill the PID
`Blocked request. This host is not allowed.`	opened the WSL IP instead of localhost	use `http://localhost:1421`, or set `VITE_ALLOWED_HOSTS=*`
`brew: command not found` in `start.sh`	macOS-only branch, Postgres was stopped	`sudo service postgresql start`, then re-run
`permission denied to create extension "vector"`	role lacks `SUPERUSER`	`sudo -u postgres psql -c "ALTER ROLE langconfig SUPERUSER;"`
`extension "vector" is not available`	pgvector package missing	install `postgresql-16-pgvector`
`could not connect to server ... :5432`	Postgres stopped (WSL restart)	`sudo service postgresql start`
`could not connect ... :5433`	`.env` still has the Docker port	set `POSTGRES_PORT=5432`
`role "langconfig" already exists`	harmless	seed is idempotent
`ModuleNotFoundError: sqlalchemy`	venv missing/stale, or wrong dir	`rm -rf backend/.venv && cd backend && uv sync`
`ModuleNotFoundError: neo4j`	old revision before the driver was declared	`git pull && cd backend && uv sync`
`bus error` during `uv sync`	repo on `/mnt/c`, or disk full, or corrupt cache	move to `~/`, `df -h ~`, `uv cache clean`
`-bash: you: No such file or directory`	pasted a macOS `WORKSPACE_ROOT=/Users/<you>/...`	use `$HOME/langconfig/backend/outputs`
seed step 2 fails on extensions	role created without `SUPERUSER`	see §4

Useful diagnostics:

```
cd ~/langconfig/backend
uv run python -c "import importlib.metadata as m; print('installed:', len(list(m.distributions())))"
uv run python -c "from dotenv import load_dotenv; load_dotenv('../.env'); from db.db_url import
        compose_db_url; print(compose_db_url())"
bash ../scripts/health_check.sh                  # postgres + API + seeded data (needs backend
        running)
```

13. Daily workflow

```
cd ~/langconfig
sudo service postgresql start     # after any WSL restart
bash start.sh                     # → :8765 API, :1421 UI
# ...
kill $(cat /tmp/langconfig-be.pid) $(cat /tmp/langconfig-fe.pid)
```

After a `git pull`: re-run `uv sync` (if `backend/pyproject.toml`/`uv.lock` changed) and `npm install` (if `package.json`/`package-lock.json` changed), then restart.

Part 2 — WSL platform notes

Everything below is WSL-specific (not needed on a plain Ubuntu server or macOS).

14. WSL tuning (`/etc/wsl.conf`, `.wslconfig`)

Enable systemd (gives you `systemctl`, and lets services start on their own):

```
sudo tee /etc/wsl.conf >/dev/null <<'EOF'
[boot]
systemd=true

[interop]
enabled=true

[automount]
options = "metadata,umask=22,fmask=11"
EOF
```

Then, **from Windows** (PowerShell): `wsl --shutdown` and reopen the terminal. Verify: `systemctl is-system-running` → `running` (or `degraded`).

Cap WSL memory/CPU from Windows at `C:\Users\<you>\.wslconfig` — WSL2 otherwise claims up to 50% of RAM, which matters because `uv sync` builds heavy wheels (torch, spacy, onnxruntime):

```
[wsl2]
memory=8GB
processors=4
swap=2GB
localhostForwarding=true      # keep true: Windows -> WSL localhost works
```

Apply with `wsl --shutdown` (all distros restart).

Other WSL habits that prevent problems:

```
git config --global core.autocrlf input     # avoid CRLF in shell scripts
sudo hwclock -s                             # fix clock drift (breaks TLS/HTTPS clones)
```

15. Windows ↔ WSL integration

Task	How
Open the UI from Windows	`http://localhost:1421` — WSL2 forwards `localhost` (needs `localhostForwarding=true`)
Open the repo in Explorer	`explorer.exe .` (from inside the repo) or `\\wsl$\Ubuntu\home\<user>\langconfig`
Edit in VS Code	install the **WSL** extension, then `code .` from the repo
Paste into the terminal	Windows Terminal: `Ctrl+Shift+V` or right-click
First backend start	Windows Firewall may prompt — allow **private** networks

> ⚠️ Do **not** work in the repo through `/mnt/c` or `\\wsl$\...` from Windows tools that rewrite files (line-ending/attribute churn). Keep the working copy in `~/`.

Reaching the app from **another device** (phone, LAN) is not supported by default: vite allows only `localhost`, and you would need the WSL IP plus a firewall rule. Start vite with `VITE_ALLOWED_HOSTS=*` and forward the port if you really must.

16. Docker on WSL (only for the optional Neo4j, §11)

Option A — Docker Desktop: install it on Windows, then *Settings → Resources → WSL integration → enable your Ubuntu distro*. `docker ps` works inside WSL afterwards.

Option B — engine inside WSL (no Desktop):

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER      # then close/reopen the terminal
sudo service docker start          # after each WSL restart (or enable systemd)
docker ps                          # verify
```

Then run the Neo4j command from §11.

17. Disk space and the WSL VHDX

Sizes that surprise people (measured on a working install):

Directory	Size
`backend/.venv`	~2 GB
`~/.cache/uv`	up to several GB
`node_modules`	~0.5 GB
Postgres data + `backend/data` uploads	grows with use

```
df -h ~                                                    # free space
du -sh ~/.cache/uv backend/.venv node_modules 2>/dev/null   # biggest consumers

# reclaim
uv cache clean                # ~/.cache/uv (re-downloads next sync)
npm cache clean --force
sudo apt clean
docker system prune -af       # if you use Docker
```

The WSL disk image (VHDX) **only grows** — deleting files inside WSL does not shrink it. To reclaim on Windows:

```
wsl --shutdown
wsl --manage Ubuntu --set-sparse true     # WSL 2.0+ (distro name from: wsl -l -v)
```

Classic alternative: `diskpart` → `select vdisk file="…\ext4.vhdx"` → `attach vdisk readonly` → `compact vdisk` → `detach vdisk`.

> Rule of thumb: keep **≥ 8-10 GB free** inside WSL before a `uv sync`; running out of space is one cause of `bus error` during install.

18. Backups and restore

`.env` holds the **Fernet master key** — back it up separately. If you lose it, every API key already stored in the database becomes unreadable.

```
# 1. secrets + config
cp ~/langconfig/.env ~/langconfig.env.bak          # or copy to /mnt/c/Users/<you>/

# 2. database (custom-format dump: compressed + restorable)
pg_dump -h localhost -p 5432 -U langconfig -Fc langconfig > ~/langconfig-$(date +%F).dump

# 3. restore (fresh machine or after a mistake)
createdb -h localhost -p 5432 -U langconfig langconfig
pg_restore -h localhost -p 5432 -U langconfig -d langconfig --clean --if-exists ~/langconfig-YYYY-MM-
        DD.dump
```

Also worth copying if you use them: `~/langconfig/backend/data/` (uploaded documents) and `~/langconfig/backend/outputs/` (generated artifacts). Neither is in git. A schema-only rebuild is also possible without a dump: `seed_all.py` + `alembic upgrade head`.

19. Starting the stack automatically

With systemd enabled (§14) — let Postgres manage itself, and optionally the app:

```
sudo systemctl enable postgresql
```

```
# /etc/systemd/system/langconfig.service
[Unit]
Description=LangConfig (backend :8765 + frontend :1421)
After=network-online.target postgresql.service
Wants=postgresql.service

[Service]
Type=oneshot
RemainAfterExit=yes
User=%i
WorkingDirectory=/home/%i/langconfig
ExecStart=/usr/bin/bash -lc 'cd /home/%i/langconfig && bash start.sh'
ExecStop=/usr/bin/bash -lc 'kill $(cat /tmp/langconfig-be.pid) $(cat /tmp/langconfig-fe.pid)'

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl enable --now langconfig@$USER.service     # note: %i = your username
```

`Type=oneshot` is deliberate: `start.sh` backgrounds uvicorn/vite and exits, so a simple service type would consider it failed.

Without systemd — Windows Task Scheduler, trigger *At log on*:

```
wsl.exe -d Ubuntu -u <user> bash -lc "cd ~/langconfig && bash start.sh"
```

Note that WSL only runs while a terminal/distro is alive; the stack stops when you `wsl --shutdown` (for an always-on server, run on a real Linux host instead).

20. Playwright browsers (optional tools)

`playwright` is a project dependency, but its browsers are downloaded separately. Only needed for the browser-automation tools:

```
cd ~/langconfig/backend
uv run playwright install chromium
# missing system libraries?
uv run playwright install-deps chromium     # uses sudo
```

21. SSH keys for git

```
ssh-keygen -t ed25519 -C "wsl-langconfig"     # Enter to accept defaults
cat ~/.ssh/id_ed25519.pub                     # paste into GitHub → Settings → SSH keys
ssh -T git@github.com                         # → "Hi <user>! You've successfully authenticated"
```

No SSH? Use HTTPS with a personal access token instead (§2).